

Centralized, Decentralized and Distributed Optimization Controls for Multi-Robot Systems using Control Barrier Functions

Alessandro Carotenuto

carotenuto.1847282@studenti.uniroma1.it

Antonio D'Orecchia

dorecchia.1403841@studenti.uniroma1.it

Filippo Giordano

giordano.2099210@studenti.uniroma1.it

January 28, 2026

Abstract

The following report collects the results of an academic project whose target is the control of a multi-agent system in unsafe conditions. The system is represented by a set of material points with the same characteristics in terms of dynamics and knowledge of the environment, which is filled with static obstacles with a known position. The agents start from different positions and move towards the same (or different) goals, with obstacles that may (or may not) be in their trajectories; as a consequence, some constraints must be applied to avoid collisions (between agents or with the surrounding obstacles) and, at the same time, to preserve connection between robots that could sense each other during their motion. To impose these collision avoidance and connectivity maintenance constraints, several optimization problems which make use of local Control Barrier Functions (CBFs) have been set up; in particular, three different strategies have been experimented: a centralized approach, which consists in a single optimization problem for the whole system; a decentralized approach, that involves an optimization problem for any single agent of the system, and a distributed approach, an improvement of the decentralized one where a global connectivity factor, independently estimated by all the agents, affects and modulates each of the optimization problems established.

Contents

Introduction	2
1 System Dynamics and CBFs	4
1.1 Dynamic model and controller	4
1.2 Control Barrier Functions	4
1.2.1 Collision avoidance CBF	5
1.2.2 Connectivity maintenance CBF	5
1.3 Optimization problems	6
1.3.1 Centralized approach	7
1.3.2 Decentralized approach	7
1.3.3 Distributed approach	8
2 MATLAB-Simulink Implementation	11
2.1 Initialization	11
2.2 SIMULINK	12
2.3 Visualization	13
3 Simulations and Results	15
4 Conclusion and Future Improvements	16
Bibliography	18

Introduction

Multi-robot systems require coordination strategies that guarantee safety, connectivity, and convergence towards shared goals. Control Barrier Functions provide a powerful framework for enforcing safety-type constraints while allowing flexible motion control: they're extremely important especially in unknown and unstructured environments, where it is considerably harder to enforce safety properties. [1]

In literature, many strategies have been adopted to control these complex systems, with or without CBFs: a centralized scheme that ensures the maintenance of the algebraic connectivity above a given value [2], a decentralized gradient-like controller based on a scalar potential function [6], a distributed implementation where each agent solves a local quadratic program (QP) and locally adapts an auxiliary variable [10]. Many works specifically focused on collision avoidance, like the backup strategy-based CBF approach described in [4], where also the concept of communication broadcasting is explained as an improvement, while other works were more interested in the problem of maintaining network connectivity, like in [9] where the agents satisfy nominal requirements that encode desired control objectives with a certain order of priority. As already said, the communication capabilities of the group are crucial to develop a system in which all the agents exchange information and move in synchronization with the surroundings; in [8], thanks to the introduction of multiple communication channels, the graph becomes more tolerant to variations in the link's quality and the failures of agents, while in [7] a local and decentralized estimation of the global connectivity is realized through a trust-based consensus algorithm. Very interesting also the adversarial setup, with a robot that chooses to not cooperate with the group in order to attempt to break or change the team's consensus, again resolved with CBFs that provide a strong network resilience. [3].

This report presents centralized, decentralized, and hybrid global-local (named also "distributed") control strategies for multi-robot systems with double-integrator dynamics based on predicted Control Barrier Functions (CBFs).

The centralized approach computes joint accelerations for all agents using a single QP, enforcing collision avoidance, obstacle avoidance, and connectivity at the network level.

The decentralized approach distributes computation across agents, each of which solves a local QP using only neighbor information. This greatly improves scalability

while retaining safety and practical connectivity.

Finally, a distributed controller augments the decentralized controller with a connectivity feedback term driven by a distributed estimate of the algebraic connectivity λ_2 . The global signal modulates local connectivity CBF gains, enabling the swarm to react collectively when global connectivity becomes weak.

The report is organized in four chapters: chapter 1 gives an overall perspective of the system, from the dynamics to the nominal control, from the different CBF instances to the optimization problems to be resolved for each strategy; in chapter 2 there is a rich explanation of the software developments (scripts, functions, simulated model), with programming details helpful to contextualize the theoretical notions of the previous chapter. All the most relevant simulations are listed and described in chapter 3, together with considerations about the pre-choices and the obtained results, and the last chapter finally summarize the work and provides some hints about possible enhancements.

Chapter 1

System Dynamics and CBFs

1.1 Dynamic model and controller

Each robot $i \in \{1, \dots, 5\}$ is modeled as a double integrator:

$$\dot{x}_i = v_i, \quad \dot{v}_i = u_i \quad (1.1)$$

where $x_i, v_i, u_i \in \mathbb{R}^2$ denote the position, velocity, and acceleration of agent i respectively. We denote by $x = [x_1^\top, \dots, x_N^\top]^\top$ and $v = [v_1^\top, \dots, v_N^\top]^\top$ the stacked state vectors.

A simple nominal controller for each agent is a PD law driving the robot toward a goal position x_{goal} :

$$u_{\text{nom},i} = -k_p(x_i - x_{\text{goal}}) - k_d v_i \quad (1.2)$$

with $k_p > 0$ and $k_d > 0$ scalar gains. The QP-based controller will then modify $u_{\text{nom},i}$ to satisfy CBF constraints.

1.2 Control Barrier Functions

A generic barrier function $h(x, v) \geq 0$ is associated to a safety condition, that has the form:

$$\dot{h} + \alpha h \geq 0 \quad (1.3)$$

where $\alpha > 0$ is typically a scalar gain.

We use predicted-distance CBFs that depend on both position and velocity through a finite prediction horizon $T_{\text{pred}} > 0$. For a pair of agents i and j , define the relative position and velocity:

$$x_{ij} = x_i - x_j, \quad v_{ij} = v_i - v_j \quad (1.4)$$

We define a predicted relative position:

$$x_{ij,pred} = x_{ij} + T_{pred}v_{ij} \quad (1.5)$$

The h function will be defined as dependent by this variable, that actually drives the optimization problems and so the evolution of the system in time.

1.2.1 Collision avoidance CBF

Concerning collision avoidance, two real situations have to be taken into account: collision among robots and collision among robots and obstacles.

In the first case, to enforce a minimum distance d_{min} (a tunable parameter) between agents i and j , we define:

$$h_{col}^{ij} = \|x_{ij,pred}\|^2 - d_{min}^2 = \|x_{ij} + T_{pred}v_{ij}\|^2 - d_{min}^2 \quad (1.6)$$

The associated CBF constraint is:

$$\dot{h}_{col}^{ij} + \alpha_{col}h_{col}^{ij} \geq 0 \quad (1.7)$$

In the second case, for a static circular obstacle o centered at x_{obs}^o (initialized in the program) with radius r_{obs}^o and safety margin r_{safe} (both tunable), given also the relative robot-obstacle state variables:

$$x_{io} = x_i - x_{obs}^o, \quad v_{io} = v_i - v_{obs}^o = v_i \quad (1.8)$$

We define:

$$h_{obs}^{io} = \|x_{io,pred}\|^2 - (r_{obs}^o + r_{safe})^2 = \|x_i + T_{pred}v_i - x_{obs}^o\|^2 - (r_{obs}^o + r_{safe})^2 \quad (1.9)$$

The corresponding CBF constraint is:

$$\dot{h}_{obs}^{io} + \alpha_{obs}h_{obs}^{io} \geq 0 \quad (1.10)$$

With an environment that contains N robots and M obstacles, there are exactly $N * (N - 1)/2$ robot-robot CBFs and $N * M$ robot-obstacle CBFs.

1.2.2 Connectivity maintenance CBF

For the connectivity aspects, the reasoning is more complex. Firstly, we should distinguish a global connectivity enforcement from a local connectivity enforcement, with the latter as a main requirement to satisfy.

A global connectivity indicator is based on the second-smallest eigenvalue λ_2 of the communication graph Laplacian:

$$L(x) = D(x) - A(x), \quad \lambda_2(L) \geq 0 \quad (1.11)$$

where A is the adjacency matrix and D is the degree matrix. The more this value is greater than zero, the more the communication graph is connected, so we could define a global CBF as [2]:

$$h(x) = \lambda_2(x) - \varepsilon \quad (1.12)$$

where $\varepsilon_\lambda > 0$ is a desired lower bound for connectivity. However, this kind of implementation is based on a very strong assumption, the assumption that each agent knows at each time the exact state of the system, fundamental to compute λ_2 and so to apply the constraint, centrally and globally. In the worst-case scenario, the agents are not aware about the whole system dynamics, so keeping a local approach is more advisable in this context.

The desired target is to avoid losing communication links, whenever and wherever they are perceived by the agents during their independent motion. Let R_{loc} be the maximum communication range, a predicted connectivity CBF is:

$$h_{\text{conn}}^{ij} = R_{loc}^2 - \|x_{ij, \text{pred}}\|^2 = R_{loc}^2 - \|x_{ij} + T_{\text{pred}}v_{ij}\|^2 \quad (1.13)$$

It's interesting to notice the opposite structure in comparison to the collision avoidance equations. The CBF inequality becomes:

$$\dot{h}_{\text{conn}}^{ij} + \alpha_{\text{conn}} h_{\text{conn}}^{ij} \geq 0 \quad (1.14)$$

This disequality keeps neighboring agents from drifting outside communication range.

Anyway, a purely local decentralized scheme may (in principle) lose the global connectivity imposition seen before, even if all local connectivity CBFs are satisfied. In practice, a distributed estimator can provide each agent with an estimate $\hat{\lambda}_2$, and with a distributed design (like the one described in the next chapter), this estimate can be used indeed to modulate the local connectivity CBF gains.

1.3 Optimization problems

In this work, an optimal solution means the resolution of a Quadratic Programming (QP) task to find the closest input u to the nominal input u_{nom} that satisfy all the linear constraints about connectivity and collisions seen before, explicated through the CBFs.

1.3.1 Centralized approach

In the centralized implementation, a single QP is solved that includes all agents' accelerations as decision variables. Given the stack of all controls into $u = [u_1^\top, \dots, u_N^\top]^\top$, the QP is realized as:

$$\begin{aligned} \min_u \quad & \|u - u_{\text{nom}}\|^2 \\ \text{s.t.} \quad & \dot{h}_{\text{col}}^{ij} + \alpha_{\text{col}} h_{\text{col}}^{ij} \geq 0, \\ & \dot{h}_{\text{obs}}^{io} + \alpha_{\text{obs}} h_{\text{obs}}^{io} \geq 0, \\ & \dot{h}_{\text{conn}}^{ij} + \alpha_{\text{conn}} h_{\text{conn}}^{ij} \geq 0 \end{aligned} \tag{1.15}$$

for all relevant agent pairs (i, j) and agent-obstacle pairs (i, o) .

This global QP enforces all constraints simultaneously with consistent coupling among all robots. Architectural scheme:

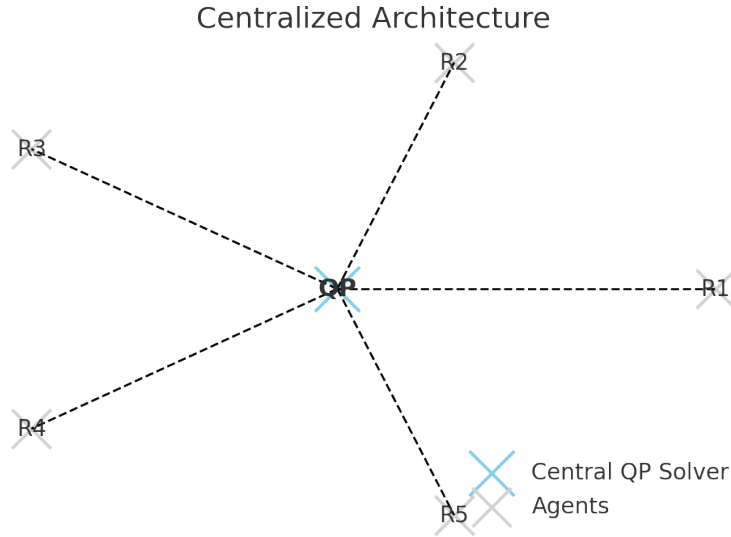


Figure 1.1: Centralized architecture: a single QP solver computes the joint accelerations for all agents.

The centralized approach yields globally consistent safety and connectivity, and is conceptually straightforward. However, the size of the QP grows with N , both in the number of decision variables ($2N$ in 2D) and in the number of pairwise constraints (on the order of N^2). This limits scalability.

1.3.2 Decentralized approach

In the decentralized implementation, each agent solves a small QP using only local information from neighbors and obstacles.

For each agent i , a neighborhood $\mathcal{N}_i$ is defined as the set of agents within a communication or sensing radius R_{glob} . Each robot has access to x_j, v_j for $j \in \mathcal{N}_i$, the local

decision variable is $u_i \in \mathbb{R}^2$ and the local QP is:

$$\begin{aligned}
\min_{u_i} \quad & \|u_i - u_{\text{nom},i}\|^2 \\
\text{s.t.} \quad & \dot{h}_{\text{col}}^{ij} + \alpha_{\text{col}} h_{\text{col}}^{ij} \geq 0, \quad \forall j \in \mathcal{N}_i, \\
& \dot{h}_{\text{obs}}^{io} + \alpha_{\text{obs}} h_{\text{obs}}^{io} \geq 0, \\
& \dot{h}_{\text{conn}}^{ij} + \alpha_{\text{conn}} h_{\text{conn}}^{ij} \geq 0
\end{aligned} \tag{1.16}$$

Here, the influence of neighbors' inputs u_j is typically approximated via zero-order hold (using their previous values). The scheme in this case will be:

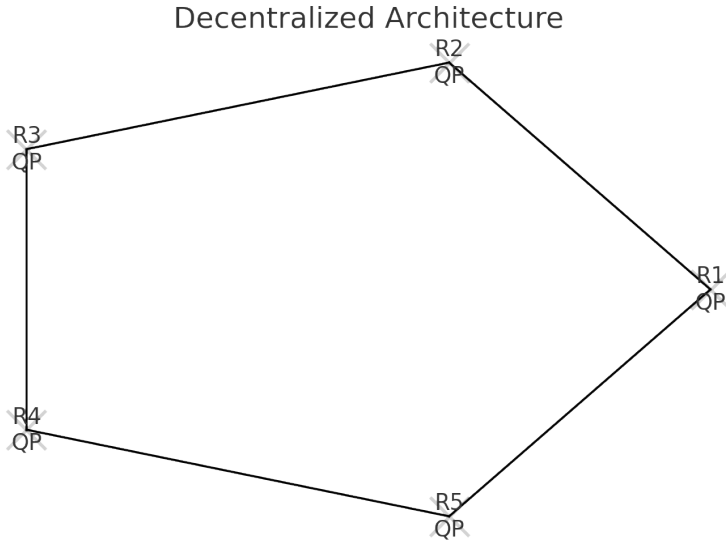


Figure 1.2: Decentralized architecture: each agent solves its own local QP using neighbor and obstacle information.

1.3.3 Distributed approach

The distributed optimization approach combines local predicted CBFs with a global connectivity feedback based on the algebraic connectivity λ_2 . [10]

Purely local connectivity CBFs enforce that neighboring robots remain within range, but they do not directly enforce a global property such as $\lambda_2(L) > \varepsilon$. A global connectivity indicator can reveal when the overall communication graph is becoming weak, even if local distances are still acceptable.

The idea is to:

- Use a distributed algorithm to estimate $\hat{\lambda}_2$;
- Modulate the local connectivity CBF gains based on $\hat{\lambda}_2$.

As said before, $\lambda_2(t)$ is the second-smallest eigenvalue of the communication graph Laplacian at time t (the algebraic connectivity). In a real system, each agent would have

an estimate $\hat{\lambda}_2(t)$ obtained from a distributed estimator. Instead of directly enforcing a CBF with this global λ_2 , we use it to modulate the local connectivity CBF gains. Defined a warning lambda λ_{warn} and a lambda gain k_λ , we call global gain factor γ_{glob} :

$$\gamma_{\text{glob}}(\hat{\lambda}_2) = \begin{cases} 1 + k_\lambda(\lambda_{\text{warn}} - \hat{\lambda}_2) & \hat{\lambda}_2 < \lambda_{\text{warn}}, \\ 1 & \hat{\lambda}_2 \geq \lambda_{\text{warn}} \end{cases} \quad (1.17)$$

To keep everything controlled, this variable is also subject to a saturation value γ_{max} .

The effective connectivity CBF gain becomes:

$$k_{\text{conn,eff}} = \gamma_{\text{glob}} k_{\text{conn}}. \quad (1.18)$$

When $\hat{\lambda}_2$ decreases toward ε_λ , the factor γ_{glob} increases, making connectivity constraints more stringent across the entire network.

The activation logic is: to avoid violent initial transients, the global feedback is activated only after the system reaches a sufficiently connected configuration. The typical logic is:

- Initially, $\gamma_{\text{glob}} = 1$ and only local CBFs are active;
- When $\hat{\lambda}_2(t)$ first satisfies $\hat{\lambda}_2(t) \geq \varepsilon_\lambda$, the global feedback is enabled;
- For t beyond that point, if $\hat{\lambda}_2(t) < \lambda_{\text{warn}}$, then γ_{glob} increases above 1, reinforcing local connectivity CBFs.

This avoids large impulsive behaviors at startup when the network might be poorly connected. Once connectivity is achieved, the controller works to keep it from degrading.

The local QP structure remains the same as in the purely decentralized case, but α_{conn} (or k_{conn}) is replaced by the effective gain $k_{\text{conn,eff}}$:

$$\begin{aligned} \min_{u_i} \quad & \|u_i - u_{\text{nom},i}\|^2 \\ \text{s.t.} \quad & \dot{h}_{\text{col}}^{ij} + \alpha_{\text{col}} h_{\text{col}}^{ij} \geq 0, \quad \forall j \in \mathcal{N}_i, \\ & \dot{h}_{\text{obs}}^{io} + \alpha_{\text{obs}} h_{\text{obs}}^{io} \geq 0, \\ & \dot{h}_{\text{conn}}^{ij} + k_{\text{conn,eff}} h_{\text{conn}}^{ij} \geq 0 \end{aligned} \quad (1.19)$$

Hence, when global connectivity weakens (small $\hat{\lambda}_2$), all agents tighten their connectivity constraints in a coherent manner, promoting a swarm-level reaction.

A simple block diagram to show how the estimations of λ_2 affect the optimization problem is:

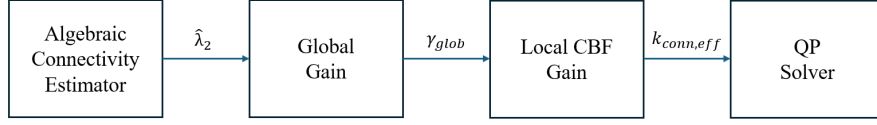


Figure 1.3: Hybrid global–local connectivity modulation. The distributed $\hat{\lambda}_2$ estimate feeds a global gain γ_{glob} , modulating the local connectivity CBF gain before entering each agent’s QP.

One limitation of this ”hybrid” global-local scheme is that it does not guarantee that λ_2 stays above ε_λ in all cases, especially if task demands (e.g., conflicting goals) are too strong. However, it provides a systematic way to react when global connectivity degrades, improving robustness in practice.

Chapter 2

MATLAB-Simulink Implementation

2.1 Initialization

The initialization purpose is to define the multi-robot scenario before any control action is applied. This phase sets up the environment, the initial conditions of the agents, and all the parameters required by the control and optimization algorithms. At the beginning, the main simulation settings are defined, including the number of robots, the duration of the simulation, and the time discretization. These parameters determine how the system evolves in time and ensure consistency between the MATLAB scripts and the Simulink models used after.

```
N      = 5;                % Number of robots
dim    = 2;                % Space dimension
dt     = 0.05;             % Timestep
Tsim   = 10;               % Tot. simulation time
steps  = floor(Tsim/dt);    % Tot. simulation
Tstop  = Tsim;             % if the model used Tstop as StopTime
colors = hsv(N);
```

Each robot is then assigned an initial position and velocity. The agents are typically arranged according to a simple geometric configuration, with the option of adding random perturbations to model uncertainty and break symmetry. Initial velocities are set to zero, so that the motion of the robots is entirely generated by the controller. The task is specified by assigning goal positions to the robots. Goals can be common to all agents or differ across the team, allowing the framework to test centralized, decentralized, and distributed control strategies under both ideal and non-ideal conditions.

```

%% -- Goal Setting --
if CONFIG.randomize_goals          % IF Randomized goal positions
    goal_region_center = [0 -1];
    goal_dispersion = 4.0;          % Controls spread of goals
    % Generate goals
    x_goal = goal_region_center + goal_dispersion * (2*rand(N,2) - 1);
    % Add outlier if enabled
    outlier_idx = -1; % Initialize

```

The environment may also include static obstacles, modeled as circular regions. Their position and size can be fixed or randomly generated. During initialization, goal locations are checked to ensure that they are not placed too close to obstacles, preventing infeasible situations at the start of the simulation. Finally, all parameters required by the control framework are defined. These include controller gains, safety margins, communication ranges, and the parameters of the predicted Control Barrier Functions. In addition, specific parameters related to connectivity monitoring and λ_2 estimation are introduced, together with mechanisms for confidence handling, decay, and robust consensus. Data structures supporting local information exchange and limited communication are also initialized, enabling the simulation and comparison of centralized, decentralized, and distributed control architectures.

```

%% --- QP setup ---

% Dict in order to parametrize the switch block in Simulink model
keys = ["Centralized", "Decentralized", "Distributed"];
values = [1, 2, 3];
d = dictionary(keys, values);

opt_strategy = "Centralized"; % choose one of the methods

```

2.2 SIMULINK

The multi-robot dynamics are implemented in Simulink, which offers an intuitive framework for modeling continuous-time systems. The robot dynamics and the control algorithms are kept separate, while Simulink ensures accurate numerical integration. FOTONTERA. So, describing this model, we have: 5 robots each robot is modeled as a double-integrator system, where the control input is the acceleration, the velocity is obtained through integration, and the position is obtained through a second integration. FOTOSUBSYSTEM. These robots are connected to 2 mux vector, called **x_vec** e **v_vec** that

unify the outputs. This signal are sent to `xlog.sim` , `xlog.sim` and the MATLAB function blocks of the three models. The first two register the temporal evolution of the states, so that we can visualize trajectories, results and animations. The other three are explain after. In the block there is also a switch that allows to change the mode, thanks to `opt_strategy`. From this blocks came out the final `u`, that goes in part, thanks to a demux in the 5 robots and into the decentralized and distributed controller.

2.3 Visualization

To better understand how the multi-robot system behaves, the simulation results are analyzed through a dedicated visualization phase. This step is essential to move beyond numerical values and directly observe how the robots interact with each other and with the environment over time. The evolution of the system is shown through an animation of the workspace, where the position and velocity of each robot are displayed together with the communication links established between neighboring agents.

```
% --- Plot evoluzione della global connectivity ---
figure;
set(gcf, 'Position', [500, 600, 600, 400])
plot(t, lambda2_log, 'LineWidth', 1.4);
xlabel('time [s]'); ylabel('\lambda_2'); grid on;
title('\lambda_2 (connectivity)');

if opt_strategy == "Distributed"
    figure;
    set(gcf, 'Position', [200, 100, 1200, 400]);

    subplot(1, 2, 1);
    plot(t, lambda_est, 'LineWidth', 1.4);
    xlabel('time [s]'); ylabel('\lambda_2'); grid on;
    title('\lambda_2 estimation per robot (no consensus)');
    legend('R1', 'R2', 'R3', 'R4', 'R5')

    subplot(1, 2, 2);
    plot(t, lambda_con, 'LineWidth', 1.4);
    xlabel('time [s]'); ylabel('y [m]'); grid on;
    title('\lambda_2 estimation per robot (with consensus)');
    legend('R1', 'R2', 'R3', 'R4', 'R5')
end
```

Obstacles and their safety margins are also visualized, making it easy to verify that collision avoidance constraints are respected throughout the motion. Alongside the animation, several plots are used to provide a clearer picture of the system dynamics. The trajectories of positions and velocities highlight how the robots move toward their goals, while the evolution of the global connectivity metric λ_2 shows how well communication is preserved during the task. In the distributed case, individual connectivity estimates are also reported, both before and after consensus, allowing the effect of information sharing and robustness mechanisms to be observed.

Chapter 3

Simulations and Results

Chapter 4

Conclusion and Future Improvements

We have presented centralized, decentralized, and distributed CBF controllers for double-integrator multi-robot systems. The centralized approach yields globally consistent safety and connectivity at the cost of computational complexity. The decentralized approach scales well but relies on purely local constraints. The distributed design introduces λ_2 -based connectivity feedback that modulates local CBF gains, enabling coherent swarm-level reactions when connectivity weakens. The λ is well-estimated by the agents by means of a communication strategy based on multi-hop and consensus. Potential ideas for further project developments could be:

- Increase the complexity of the system:
 - Dimensionality (e.g. 3D);
 - N° of elements (robots, obstacles), possibly with simplified scaling up;
 - More complicated robot dynamics (e.g. bicycle etc.);
 - More complicated task (e.g. trajectory tracking [5] etc.);
 - No knowledge about obstacle positions (e.g. SLAM application).
- Program additions:
 - Add $x-v-u$ min/max boundaries;
 - Add validation checks (e.g. wrong λ estimations etc.);
 - Add Montecarlo sampling to quantify uncertainties in λ estimates;
 - Add global connectivity CBF and compare it to the local one.
- In collision avoidance, more flexibility on the choice of the threshold (e.g. different from robot to robot, wrt obstacles etc.) or different distance function;

- In connectivity maintenance, different function for computing the weight edges (e.g. Gaussian) or different distance function;
- Integrate the project to a simulation environment (e.g. CoppeliaSIM).

Bibliography

- [1] Aaron D. Ames, Samuel Coogan, Magnus Egerstedt, Gennaro Notomista, Koushil Sreenath, and Paulo Tabuada. Control barrier functions: Theory and applications. In *2019 18th European Control Conference (ECC)*, pages 3420–3431, 2019.
- [2] Beatrice Capelli and Lorenzo Sabattini. Connectivity maintenance: Global and optimized approach through control barrier functions. In *2020 IEEE International Conference on Robotics and Automation (ICRA)*, pages 5590–5596, 2020.
- [3] Matthew Cavorsi, Lorenzo Sabattini, and Stephanie Gil. Multirobot adversarial resilience using control barrier functions. *IEEE Transactions on Robotics*, 40:797–815, 2024.
- [4] Yuxiao Chen, Andrew Singletary, and Aaron D. Ames. Guaranteed obstacle avoidance for multi-robot operations with limited actuation: A control barrier function approach. *IEEE Control Systems Letters*, 5(1):127–132, 2021.
- [5] Andrea Cristofaro, Marco Ferro, and Marilena Vendittelli. Safe trajectory tracking using closed-form controllers based on control barrier functions. In *2022 IEEE 61st Conference on Decision and Control (CDC)*, pages 3329–3334, 2022.
- [6] Paolo Giordano, Antonio Franchi, Cristian Secchi, and Heinrich Bühlhoff. A passivity-based decentralized strategy for generalized connectivity maintenance. *International Journal of Robotics Research*, 32, 01 2012.
- [7] Karlo Griparic, Marsela Polic, Marko Krizmancic, and Stjepan Bogdan. Consensus-based distributed connectivity control in multi-agent systems. *IEEE Transactions on Network Science and Engineering*, 9(3):1264–1281, 2022.
- [8] Karlo Griparić. Algebraic connectivity control in distributed networks by using multiple communication channels. *Sensors*, 21(15), 2021.
- [9] Pio Ong, Beatrice Capelli, Lorenzo Sabattini, and Jorge Cortes. Nonsmooth control barrier function design of continuous constraints for network connectivity maintenance, 2023.

- [10] Xiao Tan and Dimos V. Dimarogonas. Distributed implementation of control barrier functions for multi-agent systems. *IEEE Control Systems Letters*, 6:1879–1884, 2022.